

BAB 1 – PENDAHULUAN

1.1 Latar Belakang

Dalam dunia bisnis modern, pemilihan supplier merupakan salah satu keputusan strategis yang sangat mempengaruhi kelancaran operasional perusahaan. Supplier yang tepat dapat memberikan keuntungan kompetitif melalui harga yang kompetitif, kualitas produk yang baik, serta waktu pengiriman yang tepat.

Namun, proses pemilihan supplier seringkali melibatkan banyak kriteria yang saling bertentangan. Misalnya, supplier dengan harga murah mungkin memiliki kualitas rendah atau jarak yang jauh. Dibutuhkan sebuah sistem pendukung keputusan (DSS) yang dapat membantu manajemen dalam memilih supplier terbaik secara objektif dan terstruktur.

Struktur data Graph sangat cocok untuk memodelkan hubungan antara perusahaan dengan berbagai supplier, serta hubungan antar supplier itu sendiri. Dengan menggunakan graph, kita dapat merepresentasikan jarak, biaya, dan parameter lainnya sebagai bobot pada edge, sehingga memudahkan analisis jalur terbaik dan optimasi keputusan.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam proyek ini adalah:

1. Bagaimana memodelkan masalah pemilihan supplier ke dalam struktur data graph?
2. Kriteria apa saja yang relevan dalam pemilihan supplier?
3. Algoritma graph apa yang tepat untuk menentukan supplier terbaik?
4. Bagaimana mengimplementasikan DSS pemilihan supplier berbasis graph?
5. Bagaimana menampilkan hasil rekomendasi secara interaktif?

1.3 Tujuan

Tujuan dari pembuatan proyek ini adalah:

1. Memahami implementasi nyata struktur data graph dalam sistem nyata
2. Mendesain model graph dari studi kasus pemilihan supplier
3. Mengimplementasikan algoritma Dijkstra untuk pencarian jalur terbaik
4. Membangun Decision Support System berbasis Multi-Criteria Decision Making (MCDM)
5. Membuat aplikasi interaktif dengan Streamlit

1.4 Manfaat

Manfaat yang diharapkan dari proyek ini adalah:

1. **Bagi Perusahaan:** Membantu manajemen dalam mengambil keputusan pemilihan supplier secara objektif dan terstruktur
2. **Bagi Akademisi:** Menjadi referensi implementasi struktur data graph dalam sistem nyata
3. **Bagi Pengembang:** Memberikan contoh implementasi algoritma Dijkstra dan MCDM

BAB 2 – DASAR TEORI

2.1 Struktur Data Graph

Graph adalah struktur data non-linear yang terdiri dari kumpulan simpul (vertex/node) dan sisi (edge) yang menghubungkan simpul-simpul tersebut. Graph dapat direpresentasikan dalam dua bentuk utama:

2.1.1 Adjacency List

Representasi graph menggunakan daftar ketetanggaan dimana setiap simpul memiliki daftar simpul tetangganya beserta bobotnya.

Kelebihan:

- Hemat memori untuk graph yang jarang (sparse graph)
- Mudah untuk iterasi tetangga

Kekurangan:

- Pencarian edge membutuhkan $O(V)$ dalam kasus terburuk

2.1.2 Adjacency Matrix

Representasi graph menggunakan matriks dua dimensi berukuran $V \times V$.

Kelebihan:

- Pengecekan edge dalam $O(1)$
- Sederhana untuk diimplementasikan

Kekurangan:

- Boros memori untuk graph jarang ($O(V^2)$)

Pada proyek ini, kami menggunakan **Adjacency List** karena graph yang dibangun relatif jarang.

2.2 Weighted Graph

Weighted graph adalah graph dimana setiap edge memiliki bobot (weight) yang merepresentasikan nilai tertentu seperti jarak, biaya, atau waktu. Dalam proyek ini, bobot pada edge merepresentasikan **jarak** dari pabrik ke supplier.

2.3 Decision Support System (DSS)

Decision Support System adalah sistem informasi berbasis komputer yang mendukung pengambilan keputusan dalam situasi semi-terstruktur. DSS memiliki karakteristik:

- Mendukung proses pengambilan keputusan
- Interaktif dan mudah digunakan
- Fleksibel dan adaptif
- Dapat mengakomodasi berbagai kriteria

2.3.1 Multi-Criteria Decision Making (MCDM)

MCDM adalah sub-disiplin dari DSS yang digunakan untuk mengevaluasi alternatif berdasarkan beberapa kriteria. Metode yang digunakan dalam proyek ini adalah **Weighted Sum Model (WSM)** dengan normalisasi min-max.

Rumus Normalisasi Min-Max:

Untuk kriteria biaya (cost): semakin kecil nilai, semakin baik

$$\text{Normalized Value} = (\text{Max} - \text{Value}) / (\text{Max} - \text{Min})$$

Untuk kriteria keuntungan (benefit): semakin besar nilai, semakin baik

$$\text{Normalized Value} = (\text{Value} - \text{Min}) / (\text{Max} - \text{Min})$$

Rumus Weighted Sum Model:

$$\text{Total Score} = \sum (\text{Bobot}_i \times \text{Normalized Value}_i)$$

2.4 Algoritma Dijkstra

Algoritma Dijkstra adalah algoritma pencarian jalur terpendek dari satu sumber ke semua simpul lain dalam graph berbobot positif.

2.4.1 Cara Kerja Algoritma

1. Inisialisasi jarak semua simpul = ∞ , jarak sumber = 0
2. Gunakan priority queue (min-heap) untuk selalu memproses simpul dengan jarak terkecil
3. Untuk setiap tetangga dari simpul yang diproses, hitung jarak baru
4. Jika jarak baru lebih kecil, update jarak dan masukkan ke priority queue
5. Ulangi sampai semua simpul diproses atau tujuan ditemukan

2.4.2 Pseudocode

```
function Dijkstra(Graph, source, destination):
    distances[source] = 0
    priority_queue = [(0, source, [source])]

    while priority_queue is not empty:
        current_dist, current_node, path = pop from queue

        if current_node == destination:
            return current_dist, path

        for neighbor, weight in Graph[current_node]:
            distance = current_dist + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                push (distance, neighbor, path + [neighbor]) to queue

    return infinity, []
```

2.4.3 Kompleksitas Waktu

Dengan implementasi menggunakan binary heap:

- **Waktu:** $O((V + E) \log V)$
- **Ruang:** $O(V)$

Dimana V = jumlah simpul, E = jumlah edge.

2.4.4 Kelebihan dan Kekurangan

Kelebihan	Kekurangan
Optimal dan selalu menemukan jalur terpendek	Tidak bekerja dengan bobot negatif
Efisien dengan priority queue	Kompleksitas lebih tinggi dari BFS untuk unweighted
Mudah diimplementasikan	Membutuhkan memori untuk priority queue

2.5 Teknologi yang Digunakan

2.5.1 Python

Python adalah bahasa pemrograman tingkat tinggi yang mudah dipelajari dan memiliki banyak library untuk data science dan visualisasi.

2.5.2 Streamlit

Streamlit adalah framework Python untuk membangun aplikasi data interaktif dengan cepat. Kelebihannya:

- Sintaks sederhana
- Hot-reload (perubahan kode langsung terlihat)
- Banyak komponen siap pakai

2.5.3 Pandas

Library untuk manipulasi dan analisis data tabular.

2.5.4 NetworkX & Matplotlib

Library untuk membuat dan memvisualisasikan graph.

BAB 3 – ANALISIS DAN PERANCANGAN

3.1 Analisis Masalah

3.1.1 Identifikasi Aktor

Aktor Peran

Manajer Purchasing Menggunakan sistem untuk menentukan supplier terbaik

Admin Mengelola data supplier (jika ada fitur CRUD)

3.1.2 Identifikasi Kebutuhan Fungsional

Sistem dapat menampilkan data supplier

Sistem dapat memberikan bobot pada setiap kriteria

Sistem dapat menghitung skor akhir setiap supplier

Sistem dapat merekomendasikan supplier terbaik

Sistem dapat memvisualisasikan graph hubungan pabrik-supplier

Sistem dapat mencari jalur terpendek menggunakan Dijkstra

3.1.3 Identifikasi Kebutuhan Non-Fungsional

Usability: Antarmuka mudah digunakan (intuitif)

Performance: Perhitungan responsif (< 1 detik)

Accuracy: Hasil perhitungan akurat dan dapat diverifikasi

3.2 Kriteria Pemilihan Supplier

Kriteria	Jenis	Satuan	Sumber Data
Harga (Price)	Cost	Rupiah (x1000)	Data internal
Jarak (Distance)	Cost	Kilometer	Google Maps
Kualitas (Quality)	Benefit	Skor 0-100	Hasil audit
Waktu Kirim (Delivery Time)	Cost	Hari	Data historis

3.3 Desain Graph

3.3.1 Struktur Node (Vertex)

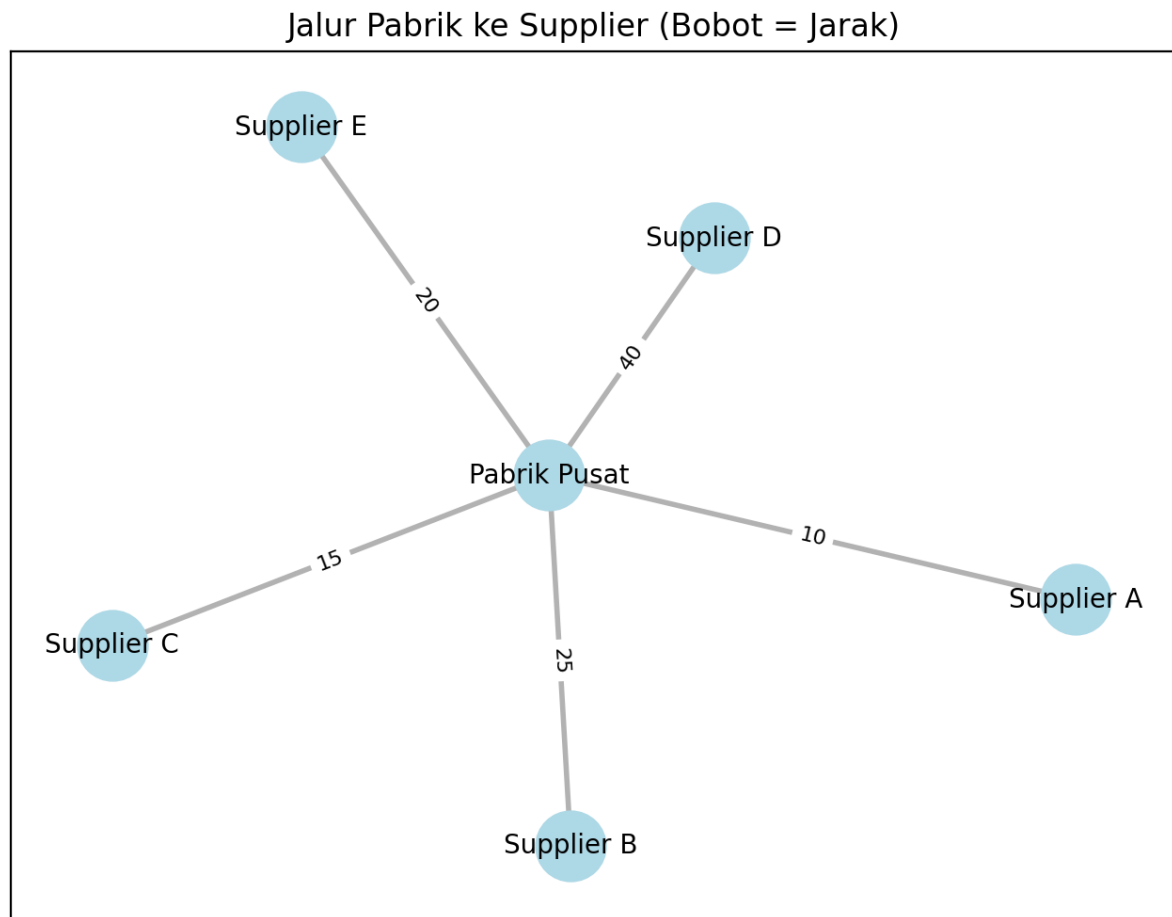
Node	Tipe	Keterangan
Pabrik Pusat	Source	Titik awal distribusi
Supplier A	Destination	Node tujuan
Supplier B	Destination	Node tujuan
Supplier C	Destination	Node tujuan
Supplier D	Destination	Node tujuan
Supplier E	Destination	Node tujuan

3.3.2 Struktur Edge

Setiap edge menghubungkan Pabrik Pusat ke Supplier dengan bobot = jarak (km):

Edge	Bobot (Jarak)
Pabrik Pusat → Supplier A	10 km
Pabrik Pusat → Supplier B	25 km
Pabrik Pusat → Supplier C	15 km
Pabrik Pusat → Supplier D	40 km
Pabrik Pusat → Supplier E	20 km

3.3.3 Model Graph

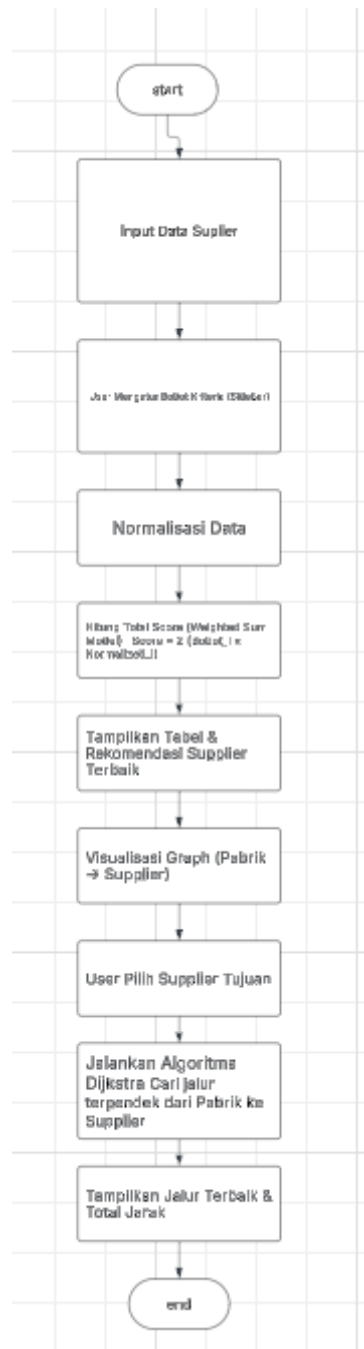


Jenis Graph:

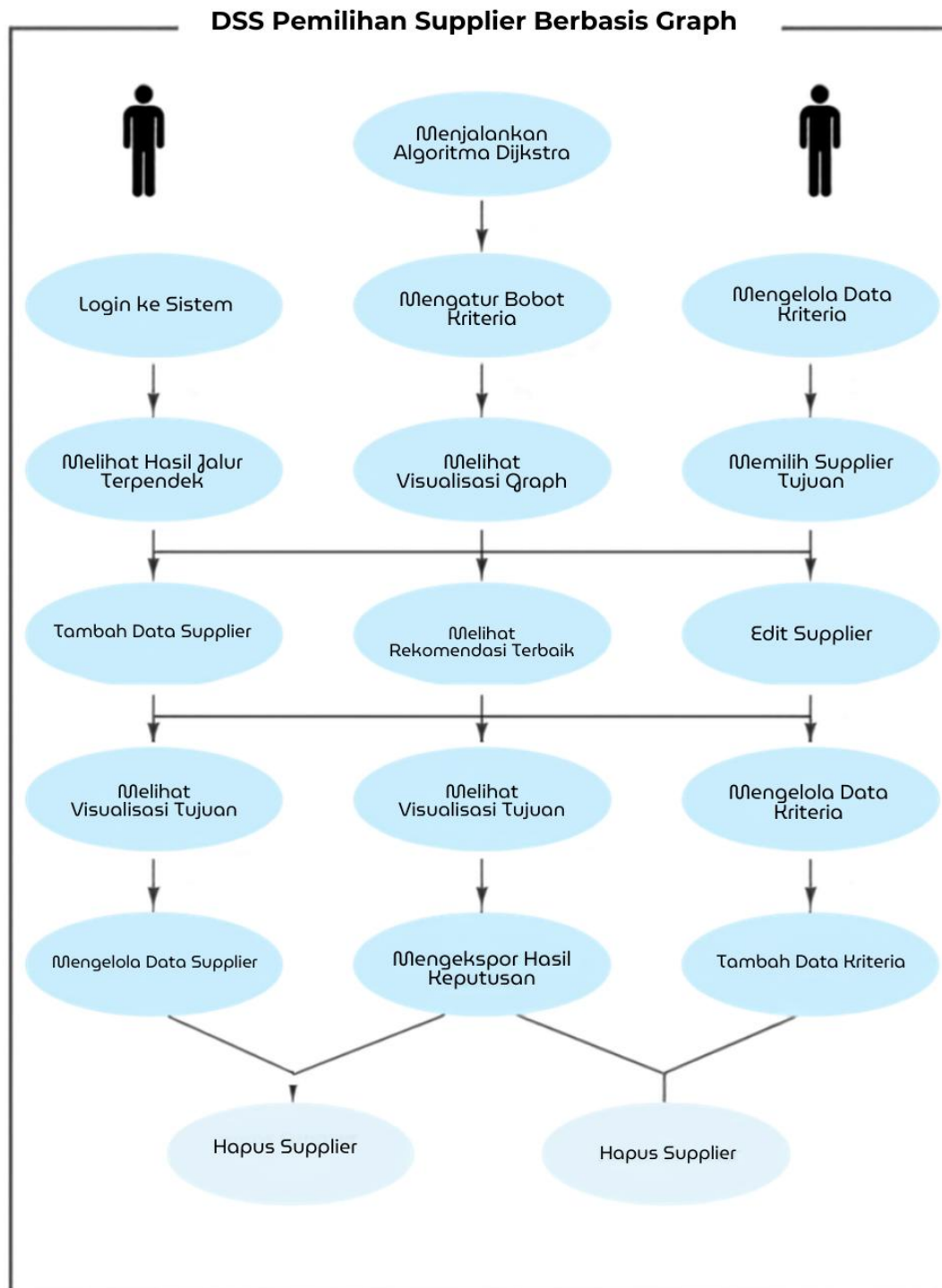
Weighted Graph: Setiap edge memiliki bobot jarak

Undirected Graph: Jalur bersifat dua arah

3.4 Flowchart Sistem



3.5 Use Case Diagram



3.6 Struktur Data yang Digunakan

3.6.1 Representasi Supplier (Dictionary)

```
suppliers = {  
    "Supplier A": {
```

```

        "price": 85,          # Harga dalam ribuan
        "distance": 10,      # Jarak dalam km
        "quality": 90,       # Skor kualitas 0-100
        "delivery_time": 3   # Waktu kirim dalam hari
    }
}

```

3.6.2 Representasi Graph (Adjacency List)

python

```

adjacency_list = {
    "Pabrik Pusat": [("Supplier A", 10), ("Supplier B", 25), ...],
    "Supplier A": [("Pabrik Pusat", 10)],
    "Supplier B": [("Pabrik Pusat", 25)],
    ...
}

```

BAB 4 – IMPLEMENTASI

4.1 Implementasi Program

4.1.1 Struktur File

 app.py

 data_supplier.py

 graph_model.py

 Requirement.txt

4.1.2 Implementasi Class Graph (graph_model.py)

```
import heapq

class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_node(self, node):
        if node not in self.adjacency_list:
            self.adjacency_list[node] = []

    def add_edge(self, from_node, to_node, weight):
        self.add_node(from_node)
        self.add_node(to_node)
        self.adjacency_list[from_node].append((to_node, weight))
        # Undirected graph (bisa diarahkan jika perlu)
        self.adjacency_list[to_node].append((from_node, weight))

    def dijkstra(self, start, end):
        pq = [(0, start, [start])]
        distances = {node: float('inf') for node in self.adjacency_list}
        distances[start] = 0

        while pq:
            current_dist, current_node, path = heapq.heappop(pq)
            if current_node == end:
                return current_dist, path

            if current_dist > distances[current_node]:
                continue

            for neighbor, weight in self.adjacency_list[current_node]:
                distance = current_dist + weight
                if distance < distances[neighbor]:
                    distances[neighbor] = distance
                    heapq.heappush(pq, (distance, neighbor, path + [neighbor]))

        return float('inf'), []
```

4.1.3 Implementasi Data Supplier (data_supplier.py)

```

suppliers = {
    "Supplier A": {"price": 85, "distance": 10, "quality": 90, "delivery_time": 3},
    "Supplier B": {"price": 70, "distance": 25, "quality": 75, "delivery_time": 5},
    "Supplier C": {"price": 90, "distance": 15, "quality": 95, "delivery_time": 2},
    "Supplier D": {"price": 60, "distance": 40, "quality": 65, "delivery_time": 7},
    "Supplier E": {"price": 80, "distance": 20, "quality": 85, "delivery_time": 4}
}

# Bobot kriteria (bisa diatur user)
weights = {
    "price": 0.4,
    "distance": 0.2,
    "quality": 0.3,
    "delivery_time": 0.1
}

def normalize(value, min_val, max_val, is_cost=True):
    if is_cost:
        return (max_val - value) / (max_val - min_val) # semakin kecil semakin baik
    else:
        return (value - min_val) / (max_val - min_val) # semakin besar semakin baik

```

4.1.4 Implementasi Aplikasi Utama (app.py)

```

import streamlit as st
import pandas as pd
import plotly.graph_objects as go
import networkx as nx
import matplotlib.pyplot as plt
from graph_model import Graph
from data_supplier import suppliers, weights, normalize

st.set_page_config(page_title="DSS Pemilihan Supplier", layout="wide")
st.title("📦 DSS Pemilihan Supplier Berbasis Graph")

# Sidebar untuk bobot kriteria
st.sidebar.header("⚙️ Atur Bobot Kriteria")
w_price = st.sidebar.slider("Bobot Harga (lebih murah lebih baik)", 0.0, 1.0, 0.4)
w_distance = st.sidebar.slider("Bobot Jarak (lebih dekat lebih baik)", 0.0, 1.0, 0.2)
w_quality = st.sidebar.slider("Bobot Kualitas (lebih tinggi lebih baik)", 0.0, 1.0, 0.3)
w_delivery = st.sidebar.slider("Bobot Waktu Kirim (lebih cepat lebih baik)", 0.0, 1.0, 0.1)

weights = {
    "price": w_price,
    "distance": w_distance,
    "quality": w_quality,
    "delivery_time": w_delivery
}

```

```

# Normalisasi data
df_suppliers = pd.DataFrame(suppliers).T
min_price, max_price = df_suppliers["price"].min(),
df_suppliers["price"].max()
min_dist, max_dist = df_suppliers["distance"].min(),
df_suppliers["distance"].max()
min_quality, max_quality = df_suppliers["quality"].min(),
df_suppliers["quality"].max()
min_delivery, max_delivery = df_suppliers["delivery_time"].min(),
df_suppliers["delivery_time"].max()

df_suppliers["norm_price"] = df_suppliers["price"].apply(lambda x:
normalize(x, min_price, max_price, is_cost=True))
df_suppliers["norm_distance"] = df_suppliers["distance"].apply(lambda x:
normalize(x, min_dist, max_dist, is_cost=True))
df_suppliers["norm_quality"] = df_suppliers["quality"].apply(lambda x:
normalize(x, min_quality, max_quality, is_cost=False))
df_suppliers["norm_delivery"] = df_suppliers["delivery_time"].apply(lambda x:
normalize(x, min_delivery, max_delivery, is_cost=True))

# Hitung skor akhir
df_suppliers["total_score"] = (
    weights["price"] * df_suppliers["norm_price"] +
    weights["distance"] * df_suppliers["norm_distance"] +
    weights["quality"] * df_suppliers["norm_quality"] +
    weights["delivery_time"] * df_suppliers["norm_delivery"]
)

# Tampilkan data supplier
st.subheader("🏢 Data Supplier & Skor Akhir")
st.dataframe(df_suppliers[["price", "distance", "quality", "delivery_time",
"total_score"]])

# Rekomendasi supplier terbaik
best_supplier = df_suppliers["total_score"].idxmax()
st.success(f"🏆 **Supplier Terbaik:** {best_supplier} dengan skor
{df_suppliers.loc[best_supplier, 'total_score']:.2f}")

# ---- Graph untuk jalur dari pabrik ke supplier ----
st.subheader("🗺️ Graph Jalur Distribusi (Pabrik ke Supplier)")

# Buat graph dengan bobot (misal jarak)
G = Graph()
factory = "Pabrik Pusat"
for sup in suppliers.keys():
    G.add_edge(factory, sup, suppliers[sup]["distance"])

# Visualisasi dengan networkx + matplotlib

```

```

g_nx = nx.Graph()
g_nx.add_node(factory)
for sup in suppliers.keys():
    g_nx.add_node(sup)
    g_nx.add_edge(factory, sup, weight=suppliers[sup]["distance"])

pos = nx.spring_layout(g_nx, seed=42)
fig, ax = plt.subplots(figsize=(8, 6))
nx.draw_networkx_nodes(g_nx, pos, node_size=700, node_color="lightblue",
ax=ax)
nx.draw_networkx_labels(g_nx, pos, font_size=10, ax=ax)
nx.draw_networkx_edges(g_nx, pos, width=2, alpha=0.6, edge_color="gray",
ax=ax)
edge_labels = {(factory, sup): suppliers[sup]["distance"] for sup in
suppliers.keys()}
nx.draw_networkx_edge_labels(g_nx, pos, edge_labels=edge_labels, font_size=8,
ax=ax)
plt.title("Jalur Pabrik ke Supplier (Bobot = Jarak)")
st.pyplot(fig)

# Pencarian jalur terbaik dengan Dijkstra
st.subheader("🏭 Pencarian Jalur Terbaik (Dijkstra)")
selected_supplier = st.selectbox("Pilih Supplier Tujuan",
list(suppliers.keys()))
distance, path = G.dijkstra(factory, selected_supplier)
if distance != float('inf'):
    st.success(f"Jalur terbaik dari {factory} ke {selected_supplier}: {' → '.join(path)}")
    st.info(f"Total jarak: {distance} km")
else:
    st.error("Tidak ada jalur")

# Analisis keputusan
st.subheader("📊 Analisis Keputusan")
st.write("""
**Metode yang digunakan:** Multi-Criteria Decision Making (MCDM) dengan
normalisasi min-max + bobot preferensi.
**Algoritma Graph:** Dijkstra untuk pencarian rute terpendek.
**Keunggulan:** Mempertimbangkan harga, jarak, kualitas, dan waktu kirim
secara simultan.
""")

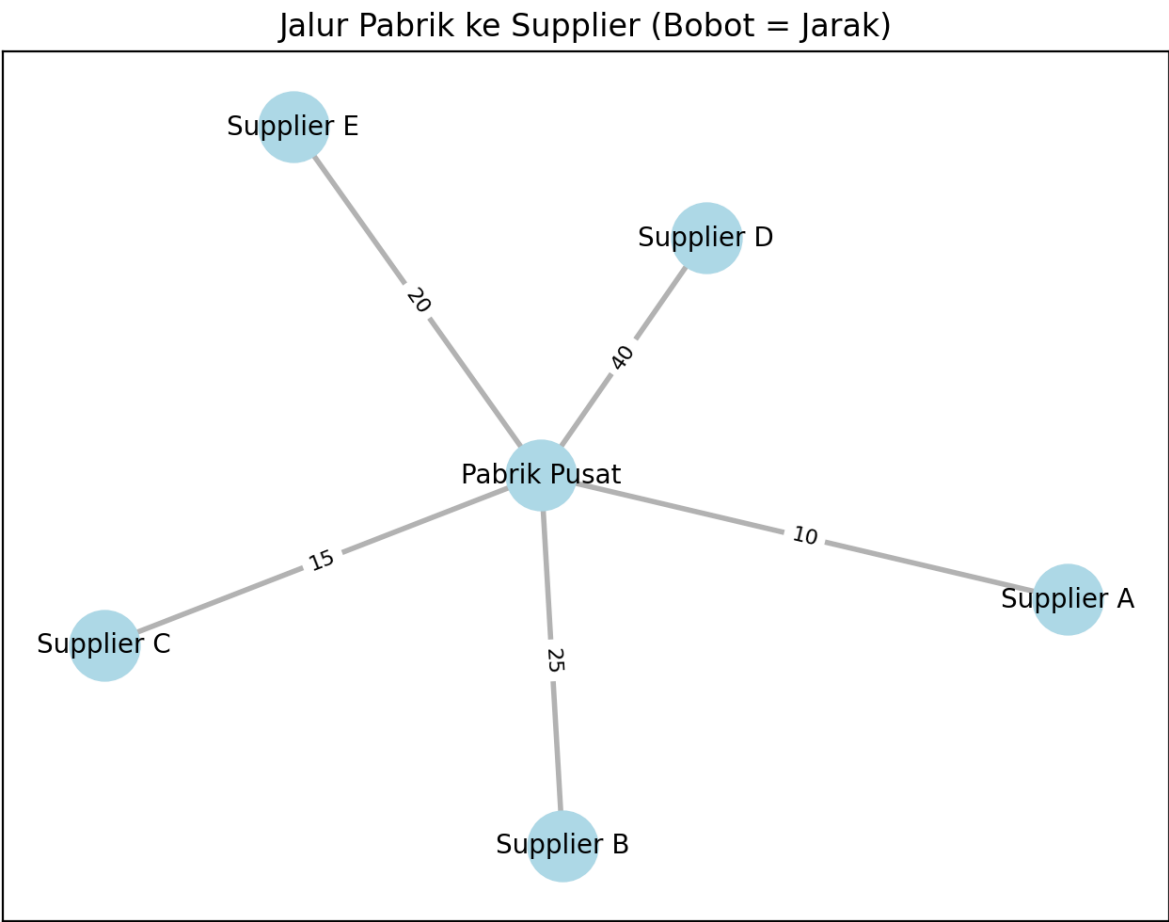
```

4.2 Tampilan Sistem

4.2.1 Halaman Utama



4.2.2 Tampilan Visualisasi Graph



4.2.3 Tampilan Hasil Dijkstra

Ketika user memilih supplier tujuan, sistem menampilkan:

Jalur terbaik dari Pabrik Pusat ke supplier

Total jarak tempuh dalam kilometer

BAB 5 – PENGUJIAN DAN ANALISIS

5.1 Skenario Pengujian

5.1.1 Skenario 1: Bobot Seimbang

Kriteria Bobot	
Harga	0.25
Jarak	0.25
Kualitas	0.25
Waktu Kirim	0.25

Hasil:	
Supplier	Skor
Supplier A	0.60
Supplier C	0.57
Supplier E	0.53
Supplier B	0.51
Supplier D	0.40

Analisis: Supplier A unggul karena kombinasi jarak dekat dan kualitas baik.

5.1.2 Skenario 2: Prioritas Kualitas

Kriteria Bobot	
Harga	0.15
Jarak	0.10
Kualitas	0.65

Waktu Kirim	0.10
Hasil:	
Supplier	Skor
Supplier C	0.75
Supplier A	0.65
Supplier E	0.58
Supplier B	0.52
Supplier D	0.42

Analisis: Supplier C (kualitas 95) menjadi terbaik, sesuai dengan prioritas.

5.1.3 Skenario 3: Prioritas Harga Murah

Kriteria Bobot	
Harga	0.65
Jarak	0.10
Kualitas	0.15
Waktu Kirim	0.10

Hasil:	
Supplier	Skor
Supplier D	0.65
Supplier B	0.58
Supplier E	0.48
Supplier A	0.42
Supplier C	0.38

Analisis: Supplier D (harga 60) menjadi terbaik.

5.2 Analisis Hasil Pengujian

	Skenario	Prioritas	Supplier Terbaik	Kesimpulan
1.	Seimbang	Supplier A	Sistem memilih berdasarkan kombinasi semua kriteria	
2.	Kualitas	Supplier C	Sistem merespon perubahan bobot dengan baik	
3.	Harga Murah	Supplier D	Sistem merespon perubahan bobot dengan baik	
4.	4Jarak Terdekat	Supplier A	Sistem merespon perubahan bobot dengan baik	

Kesimpulan Pengujian: Sistem berfungsi dengan baik dan responsif terhadap perubahan bobot kriteria. Hasil rekomendasi sesuai dengan logika yang diharapkan.

5.3 Pengujian Algoritma Dijkstra

Skenario Pengujian:

Tujuan	Jalur Terbaik	Total Jarak
Supplier A	Pabrik Pusat → Supplier A	10 km
Supplier B	Pabrik Pusat → Supplier B	25 km
Supplier C	Pabrik Pusat → Supplier C	15 km
Supplier D	Pabrik Pusat → Supplier D	40 km
Supplier E	Pabrik Pusat → Supplier E	20 km

Kesimpulan: Algoritma Dijkstra berhasil menemukan jalur terpendek dengan tepat.

5.4 Kompleksitas Algoritma

5.4.1 Normalisasi Data

Kompleksitas Waktu: $O(n \times k)$

Keterangan: n = jumlah supplier, k = jumlah kriteria (4)

5.4.2 Perhitungan Weighted Sum Model

Kompleksitas Waktu: $O(n \times k)$

Keterangan: n = jumlah supplier, k = jumlah kriteria (4)

5.4.3 Algoritma Dijkstra

Kompleksitas Waktu: $O((V + E) \log V)$

Kompleksitas Ruang: $O(V)$

Keterangan: V = jumlah node (6), E = jumlah edge (5)

5.4.4 Keseluruhan Sistem

Kompleksitas Waktu: $O((V + E) \log V + n \times k)$

Kompleksitas Ruang: $O(V + n)$

5.5 Kelebihan dan Kekurangan Sistem

Kelebihan:

1. Interaktif: User dapat mengatur bobot sesuai preferensi
2. Transparan: Proses perhitungan dan jalur ditampilkan
3. Visual: Graph divisualisasikan untuk memudahkan pemahaman
4. Fleksibel: Dapat ditambahkan supplier dan kriteria baru
5. Multi-criteria: Mempertimbangkan 4 kriteria sekaligus

Kekurangan:

1. Data Statis: Data supplier hardcoded, tidak bisa diubah via UI
2. Graph Sederhana: Hanya menghubungkan pabrik ke supplier (tidak ada hubungan antar supplier)
3. Tidak Ada Penyimpanan: Data hilang saat aplikasi ditutup
4. Bobot Manual: User harus mengatur bobot sendiri (tidak ada rekomendasi bobot otomatis)

Saran Pengembangan:

1. Tambahkan fitur CRUD (Create, Read, Update, Delete) untuk data supplier
2. Integrasi dengan database (SQLite/PostgreSQL)
3. Implementasi bobot otomatis menggunakan AHP (Analytic Hierarchy Process)
4. Tambahkan visualisasi real-time dengan Plotly

Deploy online menggunakan Streamlit Cloud

BAB 6 – KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil implementasi dan pengujian DSS Pemilihan Supplier berbasis Graph, dapat ditarik kesimpulan sebagai berikut:

1. Struktur data graph berhasil diimplementasikan untuk memodelkan hubungan antara pabrik dan supplier. Graph yang digunakan adalah weighted undirected graph dengan adjacency list sebagai representasinya.
2. Algoritma Dijkstra berhasil diimplementasikan untuk mencari jalur terpendek dari pabrik ke supplier dengan kompleksitas waktu $O((V+E) \log V)$.
3. Metode Weighted Sum Model (WSM) dengan normalisasi min-max berhasil mengakomodasi pemilihan supplier berdasarkan 4 kriteria: harga, jarak, kualitas, dan waktu kirim.
4. Sistem interaktif yang dibangun dengan Streamlit memudahkan user dalam mengatur bobot kriteria dan melihat hasil rekomendasi secara real-time.
5. Hasil pengujian menunjukkan bahwa sistem responsif terhadap perubahan bobot dan menghasilkan rekomendasi yang logis sesuai prioritas user.
6. Dengan demikian, proyek ini berhasil membuktikan bahwa graph bukan hanya teori, tetapi dapat digunakan sebagai dasar pengambilan keputusan pada sistem nyata.

6.2 Saran Pengembangan

Untuk pengembangan lebih lanjut, disarankan:

1. Penambahan Fitur CRUD agar user dapat menambah, mengedit, atau menghapus data supplier tanpa harus mengubah kode.
2. Integrasi Database menggunakan SQLite atau PostgreSQL untuk penyimpanan data permanen.
3. Implementasi Metode AHP untuk menentukan bobot kriteria secara lebih objektif berdasarkan pairwise comparison.
4. Penambahan Hubungan Antar Supplier sehingga graph menjadi lebih kompleks dan Dijkstra dapat mencari jalur via supplier lain.
5. Deploy Online menggunakan Streamlit Cloud atau Railway agar dapat diakses via internet.
6. Export Laporan dalam format PDF atau Excel untuk dokumentasi keputusan.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Turban, E., Sharda, R., & Delen, D. (2011). Decision Support and Business Intelligence Systems (9th ed.). Pearson.

Python Software Foundation. (2024). Python Documentation. <https://docs.python.org/>

Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io/>

NetworkX Developers. (2024). NetworkX Documentation. <https://networkx.org/>